

Parkinson's Disease Voice Screening Platform

Viva Examination, Technical Defense & Interview Preparation Cheat Sheet

Total Q&A Pairs: 50

Grounded In: Actual codebase implementation, real `eval_metrics.json` benchmarks, and PyTorch architecture.

Table of Contents

- 1. [Dataset & Data Handling \(Q1–Q7\)](#)
- 2. [Model Architecture & Feature Extraction \(Q8–Q15\)](#)
- 3. [Training Strategy & Validation \(Q16–Q21\)](#)
- 4. [Explainability & Attention Rollout \(Q22–Q26\)](#)
- 5. [Retrieval-Augmented Generation / RAG \(Q27–Q32\)](#)
- 6. [LLM Synthesis & Safety Guardrails \(Q33–Q38\)](#)
- 7. [Backend & System Architecture \(Q39–Q44\)](#)
- 8. [Frontend & Human-Computer Interaction \(Q45–Q47\)](#)
- 9. [Safety, Ethics, and System Weaknesses \(Q48–Q50\)](#)

1. Dataset & Data Handling

Q1: Which datasets were used to train and evaluate this platform?

A: The platform primarily utilized the **Italian Parkinson's Voice and Speech (IPVS)** dataset (65 subjects, 831 audio files covering sustained phonation `/a/`, `/e/`, `/i/`, `/o/`, `/u/`, syllable repetition `/pa-ta-ka/`, and reading passages). Additionally, cross-dataset generalization was evaluated against the **Mobile Device Voice Recordings at King's College London (MDVR-KCL)** dataset (37 smartphone continuous reading recordings).

Q2: How did you prevent subject-level data leakage between training and testing?

A: We enforced **Subject-Wise Group Stratified Splitting** (`split_manifest.json`). All audio recordings from a given patient were assigned strictly to either the `train` (45 subjects, 584 clips), `val` (10 subjects, 139 clips), or `test` (10 subjects, 108 clips) partition. At no point did the model train on one audio clip from a patient and test on another clip from that same patient.

Q3: Why is random frame-level or sample-level splitting considered fatal in medical voice ML?

A: If audio clips or frames from the same individual appear in both training and test sets, the neural network learns to recognize the acoustic identity/timbre of the speaker rather than pathology-induced vocal degradation (dysphonia). Subject-level splitting guarantees that the test set measures true diagnostic generalization to unseen patients.

Q4: What audio preprocessing pipeline is applied before feeding data to the neural network?

A: Every audio input undergoes:

- 1. Resampling to 16,000 Hz mono using `librosa.resample`.
- 2. Top-dB silence trimming at 30 dB (`librosa.effects.trim(top_db=30)`).
- 3. Amplitude peak normalization to `[-1.0, 1.0]`.
- 4. Fixed-duration segmentation to exactly 4.0 seconds (64,000 samples), using center-cropping for longer audio and repeat-padding (`pad_mode="repeat"`) for shorter audio.

Q5: Why is repeat-padding used instead of zero-padding for short phonation clips?

A: Self-supervised speech transformers (like WavLM) were pre-trained on continuous acoustic speech. Long blocks of artificial zeros create sharp step discontinuities in time-domain waveforms and spectral distortion in convolutional filters. Repeat-padding maintains the periodic acoustic harmonic structure of sustained phonation.

Q6: How does the system handle corrupt, silent, or oversized audio files?

A: `backend/app/services/audio_validation.py` enforces five sequential rejection gates returning `400 Bad Request`:

- 1. File extension validation against an allowed whitelist.
- 2. File size limit (25 MB).
- 3. In-memory SoundFile decoding with filesystem fallback.

4. Duration threshold check (≥ 0.5 seconds).

5. Acoustic energy threshold check: rejects recordings with peak $< 10^{-5}$ or RMS $< 10^{-6}$ to prevent evaluating dead microphones.

Q7: What is the cross-dataset performance on MDVR-KCL and what does it reveal?

A: On MDVR-KCL (37 English smartphone recordings), the model achieved an ROC-AUC of `0.4554` (predicting all samples as negative). This reveals significant **acoustic domain shift** caused by microphone channel mismatch (studio condenser vs. smartphone electret), linguistic differences (Italian vs. English phonology), and task variation (sustained vowels vs. continuous reading). This failure mode is explicitly documented in the system as evidence that voice models must be calibrated to specific acoustic collection protocols.

2. Model Architecture & Feature Extraction

Q8: What is the high-level architecture of the neural classifier?

A: The pipeline is a hybrid deep learning model comprising:

\$Audio (1, 64000) --[Frozen WavLM]--> (1, 199, 768) --[ConvNeXt V2 Stem]--> (1, 50, 256) --[Transformer Enc]--> (1, 50, 256) --[Attn Pool]--> (1, 256) --[MLP Head]--> Logit in $\mathbb{R}^1$

Q9: Why use WavLM-Base-Plus instead of standard MFCCs or Mel-spectrograms?

A: Handcrafted features (like MFCCs, jitter, and shimmer) capture basic acoustic properties but discard high-order temporal dynamics and phase relationships. WavLM-Base-Plus provides 768-dimensional self-supervised representations pre-trained with masked speech denoising, capturing microscopic harmonic instability, tremor, and vocal fold irregularities with high noise robustness.

Q10: Why keep the upstream WavLM foundation model frozen?

A: Freezing WavLM (94.7M parameters) prevents catastrophic forgetting on our small clinical dataset, dramatically reduces training memory overhead, eliminates gradient backpropagation through 12 heavy transformer layers, and ensures inference latency on CPU remains under 200 ms.

Q11: What is the purpose of the ConvNeXt V2 Atto-scale stem?

A: WavLM outputs a wide 199 x 768 feature map. The ConvNeXt V2 stem uses 3 stages of depthwise-separable 7 x 7 convolutions with Global Response Normalization (GRN) to compress the temporal dimension by 4x (199 $\rightarrow$ 50 tokens) while reducing channel dimensionality, acting as a lightweight convolutional adapter with minimal parameter overhead.

Q12: How many trainable parameters does the downstream classifier have?

A: Exactly **6,417,121 parameters** (verified from `model_state_dict.pt` and `eval_metrics.json`).

Q13: What does the Transformer Encoder stage do?

A: It consists of 3 Transformer Encoder layers with 4 attention heads, hidden dimension $D = 256$, FFN dimension 1024, learned positional embeddings, and dropout 0.3. It models non-local, long-range temporal dependencies across the 50 phonation tokens.

Q14: How does the Attention Pooling layer work?

A: Instead of naive mean or max pooling, Attention Pooling uses a single learnable query vector q in $\mathbb{R}^{(1 \times 256)}$. It computes scaled dot-product attention over the 50 encoder key tokens:

$\alpha = \text{softmax}(\frac{q K^T}{\sqrt{256}})$ in $\mathbb{R}^{(1 \times 50)}$

The pooled embedding is the weighted sum $v_{\text{pooled}} = \alpha V$.

Q15: How does the forward pass contract support explainability?

A: The `forward()` method of `ParkinsonsVoiceClassifier` strictly returns a tuple `(logit, attention_weights)`. The 1D attention tensor α in $\mathbb{R}^{(1 \times 50)}$ represents the exact weights assigned to each temporal segment, allowing direct visualization without gradient computation.

3. Training Strategy & Validation

Q16: What loss function and optimization setup were used during training?

A: Binary Cross-Entropy with Logits (`BCEWithLogitsLoss`), AdamW optimizer with weight decay 10^{-2} , initial learning rate 10^{-4} with cosine annealing schedule, and a dropout rate of 0.3.

Q17: What are the primary performance metrics on the held-out IPVS test set?

A: On the 108 held-out test clips (unseen subjects):

- ROC-AUC: 0.9645
- PR-AUC: 0.9712
- Expected Calibration Error (ECE): 0.0533 (10 bins)
- Accuracy: 91.67% (99/108)
- Sensitivity / Recall: 92.19% (59/64 PD detected)
- Specificity: 90.91% (40/44 Healthy correctly rejected)
- F1 Score: 0.9291

Q18: What is the operational decision threshold and how was it selected?

A: The operational threshold is 0.55 (not the default 0.50). It was determined by maximizing Youden's J statistic ($J = \text{Sensitivity} + \text{Specificity} - 1$) on the validation split. It improved test specificity from 88.64% to 90.91% without sacrificing recall (92.19%).

Q19: What does the Expected Calibration Error (ECE = 0.0533) signify?

A: ECE measures how closely predicted probabilities match empirical probabilities. An ECE of 0.0533 (5.3%) means that when the model outputs a risk score of 80%, the real-world probability of elevated risk in the test set is approximately 80% +/- 5.3%.

Q20: What are the confusion matrix numbers at threshold 0.55?

A: True Negatives (TN) = 40, False Positives (FP) = 4, False Negatives (FN) = 5, True Positives (TP) = 59.

Q21: What are the 5 False Negatives in the test set and why did they occur?

A: The 5 false negatives occurred in early-stage Hoehn & Yahr Stage 1.0 patients whose vocal symptoms had not yet manifested significant acoustic phonation stability loss, illustrating the biological limitation of voice screening in very early disease onset.

4. Explainability & Attention Rollout

Q22: What mathematical method generates the temporal attention heatmap?

A: The $N = 50$ query attention weights from the Attention Pooling layer are mapped back to the $T = 199$ physical audio frame timestamps (0.0 s to 4.0 s) using 1D Linear Interpolation, followed by min-max scaling to [0.0, 1.0] (`compute_time_aligned_attention`).

Q23: How does Attention Rollout differ from Grad-CAM or Integrated Gradients?

A: Attention Rollout directly inspects the self-attention weights learned by transformer query-key interactions during the forward pass ($O(1)$ post-processing). Grad-CAM and Integrated Gradients compute gradients of the output logit with respect to input features via multiple backward passes ($O(K)$ computation).

Q24: Did you cross-verify Attention Rollout against Integrated Gradients?

A: Yes. `compute_integrated_gradients()` is implemented in `ml/explainability/attention_rollout.py` (running 20 Riemann integration steps). Unit test `test_integrated_gradients_shape` in `ml/tests/test_explainability.py` confirms that gradient-based feature attribution correlates with attention pooling salience peaks.

Q25: What is the visual palette used for the attention heatmap and why?

A: The frontend uses Signal-Gold alpha-luminance scaling (`rgba(217, 165, 92, alpha)`). Traditional rainbow (jet/rainbow) or green-yellow-red heatmaps mislead users into interpreting high attention as "high clinical severity". Signal-gold intensity represents *algorithmic feature salience* without value-laden severity colors.

Q26: What is the scientific caveat regarding attention visualizations?

A: Attention weights reflect which temporal segments the model relied on to make its statistical prediction. They represent **mathematical relevance, not proven anatomical or causal biological proof of underlying neuropathology**.

5. Retrieval-Augmented Generation (RAG)

Q27: Why is RAG integrated into a voice screening platform?

A: Pure numerical outputs (e.g. "Probability: 0.82") are opaque and anxiety-inducing for patients, while raw LLMs frequently hallucinate medical claims. RAG grounds generated explanations in peer-reviewed clinical literature and medical guidelines (e.g., NINDS, Parkinson's Foundation).

Q28: Which embedding model and vector database are used?

A: `BAAI/bge-small-en-v1.5` (384-dimensional dense vectors via `sentence-transformers`) paired with an in-process, disk-persisted `ChromaDB` collection named `pd_knowledge_base`.

Q29: How many chunks are in the ChromaDB collection and where did they come from?

A: **21 indexed chunks** derived from 4 authoritative medical references:

1. NINDS Parkinson's Disease Neurological Overview.
2. Parkinson's Foundation Speech & Swallowing Guidelines.
3. Acoustic Biomarkers Literature on Hypokinetic Dysarthria (pitch instability, hypophonia).
4. Clinical Decision Support & Screening Limitations Guidelines.

Q30: How does the system construct semantic retrieval queries?

A: `_build_retrieval_query()` in `backend/app/services/report.py` dynamically formulates targeted domain queries:

- For elevated risk: `"vocal and speech changes in Parkinson's disease hypophonia monotone pitch articulation screening indicators"`
- For low risk: `"vocal speech health normal acoustic baseline Parkinson's screening limitations clinical evaluation"`

Q31: How many chunks are retrieved per report?

A: Top-k = 4 chunks retrieved via cosine distance.

Q32: How is the knowledge base ingested and is it idempotent?

A: Ingested via `rag/ingestion/ingest.py`. It chunks markdown files by headers (200–500 tokens with 50-token overlap) and computes SHA-256 content hashes, ensuring re-running ingestion updates only modified documents without creating duplicate vectors.

6. LLM Synthesis & Safety Guardrails

Q33: Which LLM provider and model are actively configured?

A: The platform defaults to **Groq API** running `qwen/qwen3.8-27b` (or `llama-3.3-70b-versatile`) via OpenAI-compatible endpoints. It also supports Google Gemini (`gemini-2.5-flash`), OpenRouter, and local offline Ollama (`qwen2.5:3b-instruct`).

Q34: What is the 4-part architectural separation in the clinical report?

A:

1. `model_prediction`: Raw neural network metrics (untouched by LLM).
2. `retrieved_evidence`: Verified literature passages from RAG (untouched by LLM).
3. `generated_explanation`: Bounded LLM synthesis.
4. `clinical_disclaimer`: Immutable hardcoded legal disclaimer string.

Q35: What exact phrases are on the safety blacklist?

A: Case-insensitive phrase list:

```
["you have parkinson", "you have pd", "diagnosed with", "confirmed case", "confirmed diagnosis", "positive diagnosis", "we diagnose", "has been diagnosed", "patient has parkinson", "definitely has", "diagnostic proof", "confirms parkinson"]
```

Q36: What happens if an LLM generates a prohibited diagnostic claim?

A: `_safety_check_text()` detects the forbidden phrase, logs a security warning, completely discards the LLM text, and substitutes the verified `SAFE_FALLBACK_EXPLANATION`.

Q37: What happens if the LLM provider is offline (503 / Network failure)?

A: The system degrades gracefully: `generate_report()` catches the exception and returns the verified safe fallback explanation while preserving the exact neural prediction and RAG citations.

Q38: Why is the clinical disclaimer hardcoded rather than generated by the LLM?

A: An LLM might hallucinate, shorten, or alter regulatory disclaimers. Hardcoding `CLINICAL_DISCLAIMER` as a constant in Python guarantees byte-identical compliance on 100% of generated reports.

7. Backend & System Architecture

Q39: Why choose FastAPI for the backend?

A: FastAPI provides native asynchronous request handling, automatic Pydantic request/response validation, interactive Swagger documentation ([/docs](#)), and direct in-process execution alongside PyTorch and Hugging Face pipelines without inter-process IPC serialization overhead.

Q40: What database is used and what is stored in it?

A: SQLite ([backend/app.db](#)) managed via SQLAlchemy 2.0. It stores session UUIDs, timestamps, test reference IDs, prediction labels, probabilities, threshold used, audio duration, attention heatmap JSON, and cached report JSON.

Q41: Can you prove that raw patient voice recordings are not stored on disk?

A: In [backend/app/api/predict.py](#) , the uploaded `bytes` are held in memory during the route execution. Only the derived numeric features and JSON attention vectors are passed to `repository.create_test_record()` . There is no `file.write()` or disk storage of audio in the persistence layer.

Q42: How is the backend shielded against unhandled exceptions?

A: [backend/app/main.py](#) implements a global HTTP middleware and exception handler that catches unhandled errors, logs a correlation ID (`X-Request-ID`) with stack trace, and returns a safe `500 Internal Server Error` without leaking internal paths or traceback details to the client.

Q43: How is report caching handled in the database?

A: When `POST /api/v1/report/{id}` synthesizes a report, it serializes the result into `TestRecord.report_json` . Subsequent requests (`GET /api/v1/report/{id}`) return the cached JSON in < 5 ms with zero LLM API calls.

Q44: What automated test coverage exists across the stack?

A: **116 total automated tests:**

- 79 backend and ML unit/integration tests running via `pytest` ([backend/tests/](#) and [ml/tests/](#)).
- 37 frontend component and integration tests running via `vitest` ([frontend/src/](#)).

8. Frontend & Human-Computer Interaction

Q45: What design system principles govern the user interface?

A: The UI employs a dark editorial aesthetic (`bg-void #06070C` , `bg-panel #12141F` , `signal-gold #D9A55C` , Fraunces serif display typography, Inter body). It follows the rule of "*Spend boldness in one place*"—cinematic imagery on narrative entry pages ([Home](#)), but strictly flat, high-contrast, imagery-free surfaces on utility pages ([Result](#) , [Report](#) , [History](#)) to eliminate visual competition with clinical data.

Q46: How does the interface comply with WCAG accessibility guidelines?

A:

1. All body text and badges achieve WCAG AAA contrast ratios ($\geq 8.0:1$; primary text achieves 18.36:1 against `bg-void`).
2. Risk indicators never rely on color alone; every badge pairs color with explicit text (`ELEVATED ACOUSTIC RISK INDICATED` / `LOW ACOUSTIC RISK INDICATED`).
3. High-contrast signal-gold keyboard focus rings (`*:focus-visible`) support keyboard-only navigation.

Q47: How does the browser record and preview live microphone audio?

A: [AudioRecorder.tsx](#) requests microphone access via `navigator.mediaDevices.getUserMedia` , renders a real-time oscilloscope on HTML5 Canvas via `AnalyserNode` , converts the recording into a standard 16-bit PCM `.wav` Blob using `AudioContext.decodeAudioData` , and provides an instant `<audio controls>` playback player before submission.

9. Safety, Ethics, and System Weaknesses

Q48: What is the fundamental distinction between screening and diagnosis in this project?

A: A **diagnosis** is a definitive medical determination made by a licensed neurologist incorporating physical motor exams (MDS-UPDRS), medical history, and DaTscan neuroimaging. A **screening tool** evaluates a single statistical proxy (vocal acoustic stability) to identify individuals who may benefit from comprehensive clinical evaluation. The system enforces this distinction in UI labels, API schemas, and report generation.

Q49: What are the three most significant technical weaknesses of this project?

A:

1. **Demographic & Linguistic Bias:** Training data was exclusively Italian speakers; performance drops on English continuous reading speech (MDVR-KCL ROC-AUC 0.4554).
2. **Microphone Channel Sensitivity:** Trained on high-grade condenser microphones; background noise or low-quality mobile microphones can introduce acoustic artifacts.
3. **Limited Dataset Scale:** 65 primary subjects in IPVS. While 831 clips were used, clinical diversity across Hoehn & Yahr stages was constrained.

Q50: How should this system be extended for real-world clinical deployment?

A:

1. Multi-lingual, cross-device multi-task training (combining vowel phonation, diadochokinesis, and reading).
2. Longitudinal drift tracking (monitoring how an individual patient's acoustic score changes over 6–12 months).
3. On-device tensor quantization (ONNX Runtime / INT8) for fully offline, private smartphone execution.
4. Formal IRB-approved clinical trial validation against neurologist MDS-UPDRS scores.